

北京理工大学

本科生毕业设计(论文)

指导手册

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学 院: 计算机学院
专 业: 计算机科学与技术(徐特立英才班)
班 级: 07022202
学生姓名: 胡嘉晨
学 号: 1120220611
指导教师: 陆慧梅

2026 年 北京理工大学教务部制

北京理工大学

本科生毕业设计(论文)

任务书

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学 院: 计算机学院

专 业: 计算机科学与技术(徐特立英才班)

班 级: 07022202

学生姓名: 胡嘉晨

学 号: 1120220611

指导教师: 陆慧梅

2026 年 北京理工大学教务部制

北京理工大学

本科生毕业设计（论文）任务书

学生姓名	胡嘉晨	学号	1120220611
学院	计算机学院	班级	07022202
专业	计算机科学与技术 (徐特立英才班)	题目类型	不是特殊类型毕设
指导教师	陆慧梅	指导教师 所在学院	计算机学院
题目来源	自拟题目	题目性质	软件开发
题目	Alien 内核的 x86_64 移植与 vDSO 引入		
一、题目内容 本课题旨在将组件化操作系统内核 Alien 移植至 x86-64 架构，并引入 vDSO（虚拟动态共享对象）机制。Alien 是一个基于 64 位 RISC-V 架构，具有组件化设计并支持运行时热升级功能的操作系统内核。本课题通过将 Alien 内核移植至 x86-64 平台，验证其工作在主流架构上的可行性并在其中实现 vDSO，为用户态程序提供高效的系统调用路径，避免频繁陷入内核态带来的性能开销。			
二、任务要求 1、了解操作系统内核、计算机体系结构相关背景知识，了解国内外在组件化内核等技术领域的发展趋势，构建一个基于 x86-64 架构的操作系统内核并引入 vDSO 机制，知晓和理解操作系统在安全性与可靠性上的核心要求，知晓在底层资源管理与硬件交互中需遵循的设计原则与约束，能够从性能及可维护性的角度评价 vDSO 等内核关键模块的设计。 2、在指导教师指导下，自学并掌握 Rust 编程语言、x86-64 汇编及 vDSO 实现原理。首先，完成将 Alien 内核向 x86-64 架构的移植。接下来，设计并实现 vDSO 机制，为用户态程序提供高效的系统调用路径，减少内核态陷入带来的性能开销。			

北京理工大学本科生毕业设计（论文）任务书

- 3、完成毕业设计（论文）外文翻译，锻炼跨文化交流的语言和书面表达能力，能就工程专业问题，在跨文化背景下进行基本沟通和交流。
 - 4、完成毕业设计论文，并提交可运行的内核代码、相关设计文档、测试报告及外文翻译稿。
 - 5、毕业设计开发环境
- 操作系统：开发主机为 Ubuntu 22.04。
- 目标架构：x86-64。
- 开发语言：Rust 编程语言，x86-64 汇编。
- 必备工具：Qemu 模拟器、GCC 工具链、GDB。

三、进度安排

- 1、学习 x86-64 汇编语言，调研 vDSO 的技术原理，学习 Alien 现有代码结构。（第 1 周-第 2 周）
- 2、完成 Alien 内核向 x86-64 平台虚拟机的移植工作。（第 3 周-第 8 周）
- 3、设计并实现 vDSO 机制，为用户态程序提供高效的系统调用路径。（第 9 周-第 10 周）
- 4、对移植后的内核及 vDSO 进行系统测试，优化性能。（第 11 周）
- 5、撰写并完善毕业论文，提交全部代码与文档。（第 12-14 周）
- 6、完成本科生毕业设计（论文）外文翻译。（第 1 周-第 4 周）
- 7、完成本科生毕业设计（论文）答辩。（第 15 周）

四、主要参考文献

- [1] Godones. (2025). Alien. GitHub. <https://github.com/Godones/Alien>.

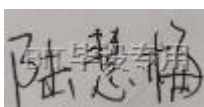
北京理工大学本科生毕业设计（论文）任务书

[2] Axboe, J. (2020). Efficient IO with io_uring.
https://kernel.dk/io_uring.pdf.

[3] Intel Corporation. (2025). Intel® 64 and IA-32 Architectures Software Developer Manuals.

[4] 赵方亮. 基于软硬协同的任务调度和中断响应研究[D]. 北京: 清华大学, 2025

五、指导教师签字:



2025 年 12 月 25 日

六、题目审核负责人意见

通过

签字:



2025 年 12 月 27 日

北京理工大学

本科生毕业设计（论文）

开题报告

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学 院： 计算机学院

专 业： 计算机科学与技术（徐特立英才班）

班 级： 07022202

学生姓名： 胡嘉晨

学 号： 1120220611

指导教师： 陆慧梅

一、选题依据

（简述该选题的研究意义和背景，国内外研究概况和发展趋势等）

传统宏内核将文件系统、网络协议栈和设备驱动等主要功能都放在同一个特权级运行，各个模块之间高度耦合。一旦某个模块出现内存错误或逻辑缺陷，故障很容易扩散到整个系统，导致崩溃或权限泄露。微内核架构则把大量系统服务移到用户态进程中，通过消息传递而不是函数调用交互，具有较强的故障隔离能力。但是，这种设计会带来频繁的上下文切换和地址空间切换，产生性能开销。Alien 内核选择了一种折中的设计方式，通过整体结构设计，让内核模块全部在内核态工作的同时，互相隔离。Alien 把内核功能拆分成多个互相隔离的故障域。每个域通过明确的接口对外提供服务，这样故障就被限制在单个域内部，不会传播到整个系统，在隔离性与性能之间取得平衡。

研究型操作系统内核 Alien 借助 Rust 语言的所有权、借用和生命周期规则，为域隔离机制提供了安全保证。每个域的实现强制使用 `safe Rust`，所有涉及底层 `unsafe` 操作的部分都被封装在可信计算基（TCB）中。例如，各域拥有独立的私有堆，其内存分配与回收由 TCB 负责，并向上层暴露为 `Box`、`Arc`、`Vec` 等 Rust 标准类型，域代码可以安全地管理堆内存。同时，所有域共享一个用于跨域通信的共享堆，其分配同样由 TCB 实现，向上提供 `DBox` 和 `DVec` 两种专用类型。域之间传递数据时，发送方在共享堆上构造对象，调用接收方接口时显式转移对象的所有权，使数据的生命周期在类型层面得到明确界定，从根本上杜绝了悬空指针和重复释放等问题。通过这种设计，绝大多数潜在的内存安全隐患被缩减至相对少量的可信计算基（TCB）中的 `unsafe` 代码，更容易进行验证。

在此基础上，本研究计划将 Alien 内核从 RISC-V 平台移植至 `x86_64` 架构并引入 `vDSO`。移植到 `x86_64` 架构的意义在于将域设计推广到 RISC-V 之外的平台并验证其可行性。`x86_64` 在服务器和桌面领域占据主导地位，拥有成熟的硬件生态和广泛的应用基础，能够为 Alien 内核提供更真实的验证环境。引入 `vDSO` 机制的意义则在于尝试如何在域设计的框架下为高频系统调用提供快速路径，在不牺牲安全性的情况下降低延迟。两项工作结合在一起，为在多架构上构建具有强隔离性和低延迟的操作系统内核提供了设计思路和实现经验。

二、研究目标和内容

（研究目标、主要内容及关键问题等）

研究目标：

本研究的目标是完成 Alien 内核从 RISC-V 到 `x86_64` 架构的移植，建立统一

的跨架构抽象层，使域设计能够适用于不同架构平台。在此基础上，引入 vDSO 机制，将时间查询系统调用使用映射到用户地址空间的只读代码与数据实现，优先使用用户态快速路径，失败后回退到系统调用慢速路径。本研究计划通过这两项工作验证 Alien 内核在多架构上的可行性，并为构建具有隔离性与低延迟的操作系统内核提供参考。

主要内容：

1. 学习 Alien 内核的域隔离架构与跨域通信机制，理解其通过域设计实现故障隔离和热设计的原理。学习 x86_64 汇编、引导流程、中断与系统调用机制，以及 vDSO 的技术原理，为后续工作奠定基础。
2. 内核移植
 - a. 引导与早期初始化：实现从 Multiboot 引导、进入汇编后配置控制寄存器切换到长模式的启动流程，然后完成 FPU/SSE 以及本地 APIC 等元件的配置。
 - b. 内存管理：实现 x86_64 四级页表结构，实现多架构统一的虚拟地址映射、权限控制与页表寄存器切换操作。
 - c. 中断与系统调用：配置中断描述符表和控制寄存器，实现 trap 与 syscall 的汇编入口及分发函数，完成用户态与内核态中断的上下文保存与恢复。
 - d. 设备探测与域加载：通过 ACPI 机制探测 x86_64 设备信息，多架构各自加载所需的域。
3. vDSO 机制引入
 - a. 编译 vDSO 源代码为位置无关的动态链接库，满足 vDSO 要求，然后生成供内核和用户库调用的 API 库，导出 vDSO 函数、数据类型等符号。

- b. 内核实现将 vDSO 二进制加载到内核地址空间，然后将 vDSO 代码段和共享数据页分别映射到各个进程的用户地址空间。
- c. 设计并实现内核刷新共享页中时间数据的逻辑。内核在固定位置持续写入新的时间数据，同时通过同步机制避免用户态在内核更新时读到脏数据。
- d. 内核通过辅助向量向用户运行时传递 vDSO 基址，后者绑定函数指针，调用时优先使用快速路径，失败时自动回退到系统调用，保证功能完整。

4. 系统测试

- a. 功能测试：设计系统调用测试，验证移植到 x86_64 的 Alien 是否通过系统调用测试并进入交互 shell，交互 shell 是否可以正常工作。同时对比 vDSO 与系统调用两条路径获取的时间值，验证两者结果是否在一定误差范围内。
- b. 性能测试：采用窗口测量法，在不同调用次数下测量两条路径的总耗时并计算单次平均开销，评估 vDSO 带来的性能提升。

关键问题：

1. 如何为 RISC-V 与 x86_64 建立统一的跨架构抽象层，将引导、页表、中断及系统调用等差异隐藏，避免大量出现条件编译？
2. 如何在维持域隔离模型的前提下，安全地将 vDSO 的代码段与共享数据页映射到用户地址空间？映射过程中需确保用户态仅能读取共享数据，只有内核拥有共享页写权限。
3. 如何设计 vDSO 共享数据同步机制，既能避免死锁问题，又能避免用户程序读到脏数据，在此基础上尽可能轻量？

三、研究方案

（拟采用的研究方法、技术路线、实验方案及可行性分析等）

研究方法：

1. 阅读相关文献和代码：通过研读中外文献与源码，学习 x86_64 汇编、引导流程、中断与系统调用机制以及 vDSO 的技术原理，阅读并分析 Alien 内核的域隔离架构与跨域通信实现。
2. 设计与实现：分模块依次设计并实现 x86_64 内核移植与 vDSO 机制，包括引导与早期初始化、内存管理、中断与系统调用建立、设备探测与域加载，以及 vDSO 的构建、映射、刷新与用户态绑定，并通过 QEMU 模拟器进行持续调试。
3. 对比测试：设计并执行功能测试与性能测试，对比 vDSO 快速路径与传统系统调用慢路径在时间查询操作上的延迟差异，定量评估 vDSO 带来的性能提升。

实验方案：

1. 实验环境：主要开发与测试环境为 Ubuntu 22.04 和 x86_64 架构 QEMU 模拟器。
2. 功能验证：测试内核能否正常启动并完成域加载，进入用户态后是否通过系统调用测试程序，交互 shell 是否正常工作，验证 vDSO 与系统调用两条路径获取的时间值一致。
3. 性能对比：
 - a. 延迟：测量 vDSO 快速路径与传统系统调用路径在 `clock_gettime` 等时间查询操作上的单次平均耗时。
 - b. 吞吐量：测试不同调用批次下两条路径的总耗时，分析缓存效应与数据稳定性。

- c. 对比标准：以移植后但未引入 vDSO 的 Alien 内核（即纯系统调用路径）作为性能对比基线。

可行性分析：

1. Alien 内核采用组件化设计，各子模块耦合度较低，为替换和扩展内存管理、中断处理等底层模块提供了良好的基础。
2. x86_64 架构的中断、系统调用、页表等核心机制均有完整的硬件手册和成熟的参考资料，架构移植有明确的技术路径可循。
3. vDSO 技术已在 Linux 内核中有成熟的跨架构实现，其设计模式和一致性问题的均有现成的解决方案可供参考。
4. 本研究的开发与验证工作可以完全在 QEMU 模拟环境中完成，环境搭建简便，资源需求较低。

四、研究计划及进度安排

第 1-2 周：熟悉 Alien 内核的域隔离架构与跨域通信机制，学习 x86_64 汇编、引导流程、中断与系统调用原理以及 vDSO 技术背景，搭建基于 QEMU 的开发和调试环境，完成整体设计方案。

第 3-8 周：在 QEMU 模拟器上完成 Alien 内核向 x86_64 平台的移植工作，依次实现 Multiboot 引导与早期初始化、物理页管理和统一页表操作实现、中断描述符表配置与 trap/syscall 入口适配，以及通过 ACPI 进行设备探测与域加载。

第 9-11 周：引入 vDSO 机制，基于现有工作构建 vDSO 和 API 库，在内核侧实现代码段和共享时间数据页的映射与权限控制，设计刷新策略，并在用户运行时库中通过辅助向量加载 vDSO 基址，实现优先 vDSO，失败时回退系统调用的逻辑。

第 12 周：对移植后的内核与 vDSO 进行功能验证和性能测试，得到结果并进行分析。

第 13-14 周：完成毕业设计论文的撰写、修改与定稿，完成外文翻译，整理全部代码与文档，准备并完成毕业设计答辩。

五、创新点及预期研究成果

创新点：

1. 在保持 Alien 内核域隔离模型不变的前提下，完成从 RISC-V 到 x86_64 架构的移植，通过将平台差异收敛到少数边界内，为内核的多架构扩展提供设计思路。
2. 在隔离内核中引入 vDSO 机制，设计时间刷新和同步机制，使时间查询操作无需系统调用即可完成，同时保留系统调用回退路径，实现系统调用的性能优化。

预期研究成果：

1. 可运行的操作系统内核：一套可在 QEMU 中运行的 Alien 内核源代码。
2. 性能测试报告：一份性能测试报告，包含协程切换延迟、异步吞吐量等对比数据，量化验证异步化改造效果。
3. 毕业设计论文：完整阐述研究工作的论文及其外文翻译稿。

六、参考文献

- [1] Godones. Alien: A simple operating system implemented in Rust[CP/OL]. [2026-05-19]. <https://github.com/Godones/Alien>.
- [2] x86_64 Contributors. x86_64: Support for x86_64 specific instructions, registers, and structures[CP/OL]. [2026-05-18]. https://docs.rs/x86_64/.
- [3] rosy233333. vdso_crate_template[CP/OL]. [2026-05-18]. https://github.com/rosy233333/vdso_crate_template.
- [4] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide[R/OL].

北京理工大学本科生毕业设计（论文）开题报告

(2026-03-31)[2026-05-19].

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.

[5] Linux Kernel Community. vdso(7) — Linux manual page[EB/OL].

(2026-03-20)[2026-05-19].

<https://manpages.debian.org/testing/manpages/vdso.7.en.html>.

七、指导教师意见

内容清晰，方案可行。同意通过！

签字：



2025 年 12 月 29 日

成绩：优(A+)，占比：0.00%

八、开题审核负责人意见

通过

签字：



2025 年 12 月 31 日

北京理工大学

本科生毕业设计（论文）

中期报告

Alien 内核的 x86_64 移植与 vDSO 引入

Alien Kernel: x86_64 Porting & vDSO Integration

学 院： 计算机学院

专 业： 计算机科学与技术（徐特立英才班）

班 级： 07022202

学生姓名： 胡嘉晨

学 号： 1120220611

指导教师： 陆慧梅

一、毕业设计（论文）主要研究内容、进展情况及取得成果

主要研究内容：

本毕业设计的核心目标是将 Alien 内核从 RISC V 移植至 x86_64 平台，建立统一的跨架构抽象层，使域隔离设计能够适用于不同硬件平台，并在此基础上引入 vDSO 机制，为高频系统调用提供快速路径。主要内容包含：

- 1、完成引导与早期初始化、内存管理、中断与系统调用适配以及设备探测与域加载等核心子系统的移植，在 QEMU 模拟器上构建可稳定运行并支持域间隔离与通信的最小 x86_64 内核。
- 2、构建符合 vDSO 规范的动态链接库，在内核中实现代码与共享数据页的用户态映射，设计内核刷新与用户读取之间的同步机制，并在用户运行时库中实现快速路径优先、系统调用回退的调用模式，最终通过功能与性能测试验证其正确性与加速效果。

进展情况：

自 3 月初至今，项目按照计划稳步推进，已完成 x86_64 架构的完整移植，并通过了多轮验证测试。具体进展如下：

第 1 周（3.2 - 3.9）：搭建 x86_64 编译环境，修改 Makefile 与链接脚本以支持 x86_64 目标，引入 `cfg(target_arch)` 条件编译框架实现架构相关代码的隔离。在 `common_x86_64` 中初步实现基于 ACPI 的设备发现，通过搜索 RSDP 并解析 SDT 表头，成功引导内核并输出 APIC、HPET 等设备信息，验证了早期启动链路的正确性。

第 2 周（3.10 - 3.16）：梳理 Arceos 与 Alien 的初始化流程，明确从汇编入口到 Rust 主函数的调用路径。完成可用 RAM 获取与物理页帧管理器的初始化，建立内核地址空间并通过简单映射测试验证页表操作的正确性。同时完成 trap 初始化，包括 GDT 中用户态与内核态段描述符的配置、IDT 中断门的设置以及 syscall 相关 MSR 的编程。

第 3 周（3.17 - 3.23）：实现基于跳板页的独立内核地址空间与 trap/syscall 处理流程，在跳板页中完成地址空间切换和栈切换。引入 `percpu crate` 并在链接脚本中添加 `percpu` 段，修改任务上下文切换逻辑以适配 x86_64 的寄存器布局。将 `domain-lib` 迁移到 x86_64，并完成三阶段设备发现：ACPI 静态表解析（获取 MADT、MCFG 等）、AML 方法执行、PCI 总线扫描，将探测到的设备统一注册为架构无关的设备类型。

第 4 周（3.24 - 3.30）：修复 trap 跳转错误及页表未映射低地址区域导致的内存访问异常，完善跳板页中用户栈保存与内核栈切换的汇编逻辑。完成三阶段

设备探测的联调，成功发现并注册 APIC、UART、VirtIO Block 和 VirtIO Net 等设备。修改 BusyBox 编译目标为 x86_64，制作对应的 initrd 镜像，分阶段集成各内核组件和驱动域。

第 5 周（4.1 - 4.7）：修复 aml crate 在解析特定 AML 方法时的报错，以及 QEMU x2APIC 模式下 timer 配置错误导致的中断异常。改进 make sdcard 流程，使用 mtools 直接操作 FAT 镜像以避免 sudo 权限。修复 UART 域空输出和 syscall 域在 x86_64 下的参数传递差异，添加线程局部存储支持。最终成功进入交互式 Shell，可运行 ls、echo 等基本命令，验证了系统调用、任务切换和文件系统等核心功能的正确性。

取得成果：

- 1、实现了 x86_64 引导与基础运行环境，内核能够在 QEMU x86_64 虚拟机上从 Multiboot 引导，输出详细初始化日志，并稳定运行。
- 2、实现了物理内存管理与地址空间，实现了物理页帧分配器和基于跳板页的独立内核地址空间，支持页表动态切换。
- 3、实现了中断与系统调用基础设施，正确配置 GDT、IDT、syscall MSR，trap 处理正常，系统调用路径打通。
- 4、实现了设备发现机制，通过三阶段 ACPI+AML+PCI 探测，成功识别 APIC、UART、VirtIO 块设备和网络设备，为驱动加载提供统一设备列表。
- 5、进入了用户态执行环境，成功加载 initrd 中的 BusyBox 程序，进入 Shell 并执行基本命令，证明系统调用、任务切换和文件系统基础功能正常。

Alien 操作系统在 x86_64 架构上的移植已基本完成核心功能，达到了中期预期目标。

二、存在的问题和拟解决方案

尽管 Alien 内核在 x86_64 架构上已成功进入 Shell 并执行基础命令，但在测试与集成过程中仍发现若干问题，需要在下一阶段解决。

1. 用户测试程序权限错误(shell 可能出现 Permission Denied)

当前内核能够进入 Shell 并执行 ls、echo 等 BusyBox 内置命令，但执行 tests 文件夹内的独立测试程序时出现 permission denied 错误。初步排查表明，该问题

并非文件系统权限位（如 0755）配置错误，可能与内核在加载 ELF 文件时的权限检查、文件系统接口实现有关。

拟解决方案：

- 1、从 ELF 解析器开始，跟踪测试程序的加载过程，确认内核是否正确设置了代码段、数据段的内存页属性（可读、可写、可执行）。
- 2、检查 `exec` 系统调用的实现路径，对比 RISC-V 版本的权限检查逻辑，找出 `x86_64` 迁移过程中可能遗漏的条件。
- 3、在文件系统层添加调试日志，输出 `inode` 的权限位，验证访问控制判断是否正确。
- 4、可先放宽权限检查（如直接允许 `root` 执行），待后续完善安全模型后再恢复严格校验。

2. 尚未对外部设备支持完全

虽然已通过 ACPI、AML 和 PCI 探测成功发现 APIC、UART、VirtIO Block 和 VirtIO Net 等设备，并注册为统一的 `CommonDeviceType`，但目前仅有 UART、VirtIO Block、VirtIO Net、APIC 驱动完全工作，VirtIO Input、VirtIO GPU、HPET 等设备还未在 `x86_64` 中支持。外部设备的驱动覆盖和功能集成仍不完整。

拟解决方案：

- 1、优先完成 VirtIO 块设备驱动的注册与测试，将其与现有的 VFS 层关联，使内核能够从磁盘镜像中读取文件，从而摆脱完全依赖 `initrd` 的限制，实现更灵活的存储方案。
- 2、借鉴 RISC-V 版本的驱动框架，将 `CommonDeviceType` 与 `drivers` 域中的设备驱动进行匹配绑定，实现自动探测与加载，尽可能复用已有代码。
- 3、完善 APIC 中断路由，为每个外设分配正确的 IRQ 号，并在设备驱动中注册中断处理函数，验证中断响应的正确性。
- 4、分阶段验证：先完成串口（已工作）和时钟中断（HPET 或 PIT），再扩展到 VirtIO 块设备和网络设备，最后考虑键盘、鼠标等输入设备。

3. 未移植 VDSO 模块

当前系统调用通过 `syscall` 指令和 MSR 配置实现快速陷入，但未实现 VDSO 机制。目前已有现成的 `vdso crate`

（https://github.com/rosy233333/vdso_crate_template）可供参考和集成。

拟解决方案：

- 1、阅读并理解 `vdso_crate_template` 的实现，掌握 VDSO 在 `x86_64` 上的工作原理：如何构建 VDSO 页面、如何将其映射到用户地址空间、用户态程序如何找到 VDSO。
- 2、选择 1-2 个适合 VDSO 加速的系统调用（例如 `gettimeofday`）进行实现，验证性能提升效果与接口兼容性。
- 3、将 VDSO 模块作为独立的 `crate` 集成到 `Alien` 中，确保其与现有系统调用路径（`syscall` 指令）共存且互不冲突。

三、下一步研究任务与进度安排

第 0 周（中期） - 第 1 周：修复驱动与用户程序支持

解决用户测试程序的权限错误，完善 `VirtIO` 块设备与 `VFS` 的绑定、网络设备驱动的基础收发功能，完成 `APIC` 中断路由配置。确保所有用户测试用例在 `x86_64` 上全部通过，为 `vDSO` 的集成提供稳定的系统基础。

第 1 周 - 第 3 周：引入 `vDSO` 机制

基于 `vdso_crate` 构建 `vDSO` 生成工具，编译位置无关的动态链接库并生成 API 库，导出 `clock_gettime` 与 `gettimeofday` 等函数符号。在内核侧完成 `vDSO` 代码段与 `vVAR` 共享时间数据页的映射与权限控制，设计与实现刷新策略。在用户运行时库中通过辅助向量获取 `vDSO` 基址并绑定函数指针，实现快速路径优先、系统调用回退的工作模式。

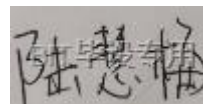
第 4 周 - 第 8 周：测试与论文撰写

对 `vDSO` 进行功能测试与性能测试，比较快速路径与系统调用路径的时间一致性和延迟差异，采用窗口测量法统计单次平均开销与加速比并完成针对性优化；同时进行全系统功能验证。撰写毕业设计论文（包含 `x86_64` 移植、`vDSO` 设计与实现等章节），制作答辩 PPT 并准备演示。5 月 1 日前完成所有开发工作，随后进入论文打磨与答辩准备。

四、指导教师意见

符合既定的进度要求。

签字：



2026年4月20日

成绩：优(A+)，占比：0.00%

五、中期审核负责人意见

签字：



2026年4月22日

毕业设计（论文）指导教师评语表

指导教师对毕业设计（论文）的评语：

选题针对具有更高容错和更小的漏洞影响范围的Alien内核操作系统目前只能运行在RISC-V平台上的问题，扩大其适用范围，具有一定的应用价值，符合复杂工程问题要求。论文介绍了微内核和隔离研究等相关领域研究现状。

论文分析了 x86_64 平台和屏蔽多架构差异的需求，设计并实现了 Alien 在 x86_64 上的启动流程、trap/syscall 入口、内存管理及设备探测与驱动加载框架，在此基础上，进一步设计并集成了 vDSO 机制。最终在 QEMU 平台上完成了功能验证和性能测试，结论可信。

胡嘉晨同学工作认真，有很好的独立实践能力，按照 OBE 标准，较好地完成了毕业设计各指标点要求和毕业设计全过程。

论文结构合理，引用规范，设计方案恰当，有一定难度和工作量，过程规范，表明学生具有扎实的专业基础知识和综合运用能力，已具备独立工程实现能力，论文达到本科毕业设计对应的毕业要求，同意参加论文答辩。

是否有校外指导教师：☐是 ☒否；若选择“是”，校外指导教师对毕业设计（论文）的评语：

北京理工大学本科生毕业设计（论文）评语表

成绩：优(A+), 占比 0.00%

指导教师：



2026年5月26日

北京理工大学

毕业设计（论文）匿名评阅评语表 1

学生姓名	胡嘉晨	学号	1120220611			
学院	计算机学院	班级	07022202			
专业	计算机科学与技术（徐特立英才班）					
指导教师	陆慧梅	指导教师 所在学院	计算机学院			
题目	Alien 内核的 x86_64 移植与 vDSO 引入					
评阅结果	良 (A)，占比 0.00%					
评语：（请参考评议要素，给出评语，并指出论文存在的不足及建议，评分为 C 和 D 的必须有明确的修改意见，不少于 200 字。） 论文选题面向传统宏内核高度绑定导致的可靠性问题，以 Alien 操作系统的 x86_64 架构移植与 vDSO 引入为切入点，具有很高的工程挑战、理论价值与实践意义。论文设计了跨架构的抽象接口，技术路线十分清晰，创新意识较好。 不足之处主要体现在以下几个方面：第一，相关工作的综述在探讨隔离设计时相对宏观，对现有其他使用 Rust 构建的多架构操作系统（如业界或学术界同类前沿工作）的横向详细比较不够充分；第二，文字表达总体专业流畅，但部分代码片段的展示排版、图表注释细节及参考文献格式仍有进一步规范的空间；						
评阅人： 2026 年 5 月 24 日						

北京理工大学

毕业设计（论文）匿名评阅评语表 2

学生姓名	胡嘉晨	学号	1120220611
学院	计算机学院	班级	07022202
专业	计算机科学与技术（徐特立英才班）		
指导教师	陆慧梅	指导教师 所在学院	计算机学院
题目	Alien 内核的 x86_64 移植与 vDSO 引入		
评阅结果	良 (A)，占比 0.00%		

评语：（请参考评议要素，给出评语，并指出论文存在的不足及建议，评分为 C 和 D 的必须有明确的修改意见，不少于 200 字。）

选题针对传统宏内核耦合度高、故障易扩散，以及高频系统调用开销大的问题，具有一定的研究意义与工程应用价值，论文梳理了域隔离内核、Rust 内存安全、vDSO 优化等相关领域研究现状。

论文分析了 Alien 内核 x86_64 移植与 vDSO 引入的技术需求，设计并完成了 Alien 内核 x86_64 架构适配系统，实现了启动流程、中断与系统调用入口、内存管理、设备探测框架的架构适配，集成了 vDSO 快速路径机制，完成功能验证与性能测试，测试数据详实，结论可信。

论文结构完整，章节逻辑清晰，引用格式规范，移植设计与优化方案合理，工作量比较饱满且具备一定技术难度，研究过程规范，表明学生具有扎实的操作系统底层开发、Rust 编程等专业基础知识和工程实践能力，已具备独立工程实现能力，论文达到本科毕业设计对应的毕业要求，同意参加论文答辩。

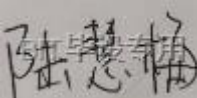
北京理工大学本科生毕业设计（论文）评语表

评阅人：

2026 年 5 月 23 日

北京理工大学

毕业设计（论文）答辩评语表

答辩委员会（小组）组长			
姓名		职称	签字
段春晖		预聘副教授	
答辩委员会（小组）组员	姓名	职称	签字
	陆慧梅	副教授	
	余皓然	长聘副教授	
	段春晖	预聘副教授	
	黎有琦	助理教授/特别副研究员	
	韩峰	工程师	

答辩中提出的主要问题及回答的简要情况：

Q1: Alien 有什么优势，为什么要移植到 x86_64?

A1: Alien 用 Rust 语言来保证安全设计。Alien 把内核功能按职责拆分成域，域之间通过公开接口交互，功能和边界划分明确。域的实现强制使用 safe Rust，底层的 unsafe 操作由 TCB 封装成安全接口，保证域自身的内存安全。在这个基础上，内核加载域时会自动设置故障隔离代理和热升级代理，用来捕获错误和执行运行时升级，避免错误泄漏到其它模块，也方便开发者维护域的实现。Alien 原本只支持 RISC-V，移植到 x86_64 能将此工作从 RISC-V 扩展到多架构，并检验域隔离设计在 x86_64 上的可行性，然后引入 x86_64 的若干特性。

Q2: 为什么用 RISC-V 的 Alien 作为后续 x86_64 移植的基础，而不是直接重

北京理工大学本科生毕业设计（论文）评语表

写一个新系统？

A2:因为 Alien 的核心机制已经把功能、接口和边界整理成了相对稳定的框架，迁移时可以优先复用域机制、共享堆通信和域管理逻辑，只把平台相关部分收敛到启动、异常、设备和内存抽象上。这样既能降低移植成本，也能验证原有隔离设计在 x86_64 上的可行性。

Q3:原有的 RISC-V 路径是否受到影响？

A3: 原有的 RISC-V 实现仍正常工作，与 x86_64 侧一样引入了 vDSO，并验证了功能正确

答辩委员会（小组）代表（签字）：

段春晖

2026 年 5 月 27 日

北京理工大学本科生毕业设计（论文）评语表

答辩委员会（小组）的评语：

论文选题符合复杂工程问题标准和 OBE 中毕业设计的毕业要求，论文分析了 Alien 内核缺乏 x86_64 平台适配、无 vDSO 快速调用机制、难以实现多架构扩展的适配优化需求，设计完成了基于协程与用户态中断的异步化 Alien 内核系统，搭建了跨平台抽象层，在保留原有 RISC-V 平台全部功能的基础上，完成 x86_64 平台内核全套基础功能移植，实现 vDSO 机制与用户态快速系统调用路径，完善内核异步化运行能力，并进行了 QEMU x86_64 仿真平台环境测试，采用基本原理正确，设计合理，结论可信。论文阐述清楚，撰写符合规范，结构合理，达到本科水平。答辩中问题回答正确。软件验收运行较好。表明该生已较好掌握计算机的基础知识和理论，达到了毕业设计对应的毕业要求的各项指标点要求，完成了毕业设计任务，答辩委员会一致同意通过论文答辩。

答辩委员会（小组）代表（签字）：

段春晖

2026 年 5 月 27 日

答辩委员会（小组）给定的成绩：良(A)，占比 100.00%

答辩委员会（小组）主任（签字）：

段春晖

2026 年 5 月 27 日

北京理工大学本科生毕业设计（论文）评语表

最终成绩：良(A)

成绩构成及占比：

开题成绩：优(A+), 占比：0.00%

中期成绩：优(A+), 占比：0.00%

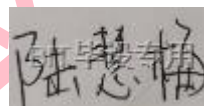
外文翻译成绩：优(A+), 占比：0.00%

指导教师评阅成绩：优(A+), 占比：0.00%

匿名评阅成绩：良(A), 占比：0.00%

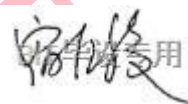
答辩成绩：良(A), 占比：100.00%

指导教师签字：



2026年5月27日

责任教授签字：



2026年5月27日

毕业设计（论文）开始日期 2025年12月25日

截止日期 2026年6月2日

毕业设计（论文）答辩日期 2026年5月27日

北京理工大学